

① Explain the following terms.

1.1 Conflict instructions

↳ Let us consider a schedule S in which are two consecutive instructions, I and J , of transactions T_i and T_j , respectively ($i \neq j$). If I and J refer to the same data item, then we cannot swap I and J . Since the swapping may affect the result of instructions in the schedule. Namely, if I and J refer to the same data item Q , then the order of the two step matters. Since we are dealing with only read and write instructions. There are 4 cases we need to consider:

1 $I = \text{read}(Q)$, $J = \text{read}(Q)$ order of I and J doesn't matter, since the same value of Q is read by T_i and T_j regardless of the order

* 2 $I = \text{read}(Q)$, $J = \text{write}(Q)$ order of I and J matters

* 3 $I = \text{write}(Q)$, $J = \text{read}(Q)$ order of I and J matters

* 4 $I = \text{write}(Q)$, $J = \text{write}(Q)$ order of I and J matters

We say that I and J conflict if they are operations by different transactions on the same data item, and at least one of these instructions is a write operation.

e.g.

T_1	T_2
read(Q)	
	write(Q)
write(Q)	↗ conflict, can't swap

1.2 Conflict serializable schedules

↳ We say that schedule S is conflict serializable if it is conflict equivalent to a serial schedule, where conflict equivalent schedule means a non-conflict instructions within a schedule can be swap and transform to one another

e.g.

T_1	T_2		T_1	T_2
read(A)		=	read(A)	
write(A)			write(A)	
	read(A)		read(B)	
	write(A)		write(B)	
read(B)				read(A)
write(B)				write(A)
	read(B)			read(B)
	write(B)			write(B)

serial schedule

1.3 non-recoverable schedule $\rightarrow$ source doesn't commit before the reader commit

T ₆	T ₇	$\rightarrow$ T ₇ commits while T ₆ is still in the active state. Now
read(A)		Suppose that T ₆ fails before it commits. T ₇ has read the
write(A)		value of data item A written by T ₆ . Therefore, we say
	read(A)	that T ₇ is dependent on T ₆ . Because of this, we must
	commit	abort T ₇ to ensure atomicity. However, T ₇ has already
read(B)		committed and can't be aborted. Thus, we have a situation
		where it is impossible to recover correctly from the failure of
		T ₆

1.4 Write-Ahead Logging

- $\rightarrow$ requires the undo information in the log has been output to stable storage, and it permits the redo information to be written later. The difference is relevant in system where undo information and redo information are stored in separate log records
- $\rightarrow$ log record must be output to the log files (stable storage) before the corresponding modified database buffer is output to DBspace

1.5 Shadow Paging

- $\rightarrow$ is an alternate to log-based recovery, the scheme is useful if transaction executed serially
- $\rightarrow$ can provide instant recovery
- $\rightarrow$ maintain two page tables during the lifetime of a transaction:
 - 1) current page table
 - 2) shadow page table
- $\rightarrow$ store a shadow page in a non-volatile storage, such that the state of the database prior to transaction execution may be recovered
- $\rightarrow$ current page table is used for data item accesses during execution of a transaction

↳ Whenever any page is about to be written for the first time

- a copy of this page is made onto an unused page
- the current page table is then made to point to the copy
- an update is performed on the copy

↳ commit:

1) Finish all modified pages in main memory to disk

2) output current page table to the disk

3) make current page table the new shadow page table, as follow:

- keep pointer to the shadow page table at a fixed (known) location on a disk
- update the pointer to point to current page table on the disk

(2) Show a schedule which gives correct results but is not view serializable

T1	T2	T1	T2
UPDATE ACCT		read(A)	
SET amount = amount - 50		A := A - 50	
where Acc# = 'A'		write(A)	
	UPDATE ACCT		read(B)
	SET amount = amount - 10		B := B - 10
	where Acc# = 'B'		write(B)
UPDATE ACCT		read(B)	↙ can't swap
SET amount = amount + 50		B := B + 50	
where Acc# = B		write(B)	
	UPDATE ACCT		read(A)
	SET amount = amount + 10		A := A + 10
	where Acc# = 'A'		write(A)

③ database recovery problem, ^{how} log-based solve the problem, AIS advantages & disadvantages

Database recovery problem statements

① There are committed transaction whose modified buffer values are not yet output to the DBSpace on the disk.

② There are uncommitted transactions whose modified buffer values are already output to the DBSpace

How log-based recovery solves the problem?

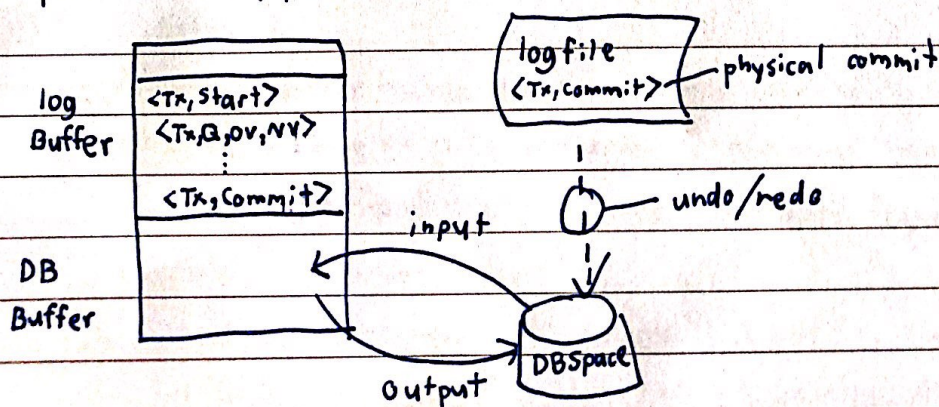
step 1: The recovery manager (module) of the DBMS will scan the logfile from the back; to determine which transaction is committed and which transaction is failed

step 2: Transaction id of committed transactions will be kept in a list called "redo" list. A transaction id of failed transactions will be kept in "undo" list

step 3: For each transaction in the undo list, old values are applied to the DBSpace. Undo the modified values of uncommitted transactions which solves problem statement ②

step 4: For each transaction in redo list, committed transactions, new values are applied. Solves problem statement ①

More: The content of a logfile will be written in the DBSpace when the step ③, ④ take place in recovery process.



SUBJECT: NO: DATE:/...../.....

③ (cont.) log-based recovery without BLS → AIS only!

AIS: how it works?

COMMIT: Normal commit process

rollback: without the old value we should not let the DB modification take place before the transaction commit (output DB Buffer to DB Space after commit only).

Namely, you don't have to undo because you didn't do any modification until commit, because until it is committed log buffer can't go to DB Space

Advantages: fast commit server.

no space required for BLS

no BLS full problems

Good for small transaction, online transaction, work group server

Disadvantages: not suitable for large transaction because DB buffer could be full before commit

④ ACID properties

Atomicity: Either all operations of the transaction are properly reflected in the database or none are.

Consistency: Execution of a transaction in isolation preserves the consistency of the database

Isolation: Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions.

Intermediate transaction result must be hidden from other concurrently executed transactions.

That is, for a pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished

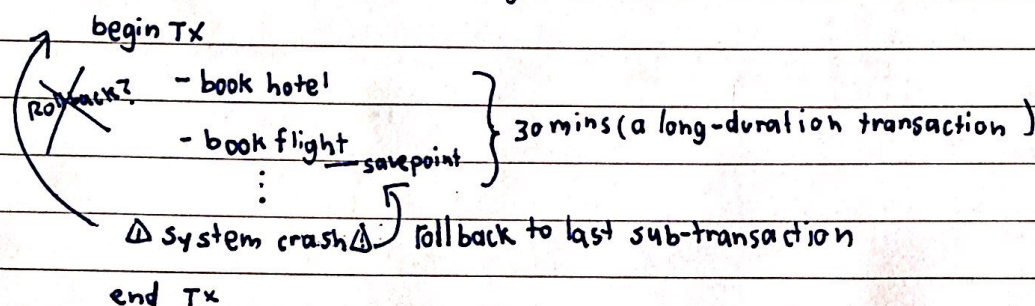
Durability: After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Double A

④ (cont.) exceptions

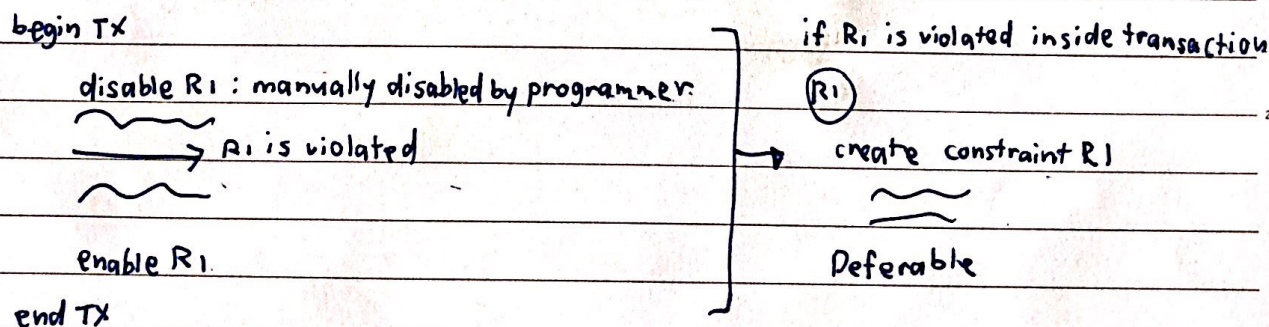
Atomicity

- In case of system failure, in certain conditions we want the system to continue from its point of failure → disregarding atomicity
- This way we can retry certain steps and not the whole process, also called nested transaction which uses to handle long-duration transaction



Consistency

- In distributed transactions, two-phase commit (local and global commit) → disregarding consistency in most cases
- If a data correctness and consistency is the main issue → a centralized database is a must if we can tolerate eventual consistency



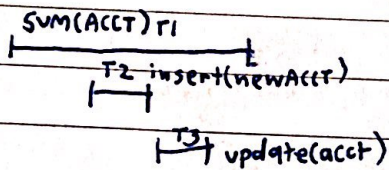
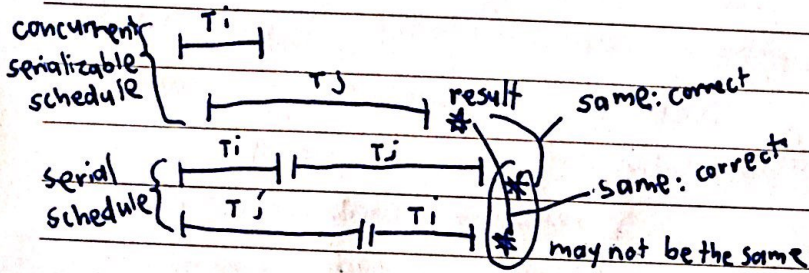
Isolation

- In case we need transaction to be arranged sequentially → disregard isolation
- use concurrent schedule and serial schedules → combined can have concurrent serializable schedule which is preferred

SUBJECT:

NO:

DATE:

 T_1 should not see T_2 & T_3

Durability

↳ changes are only write back from commit/rollback from the main memory to the disk when that file is no longer busy → disregarding atomicity

⑤ SQL Levels of Consistency? correctness property of each level, problems that it solve,

Levels of Consistency

1) Serializable (default) : Lock type: 1, Lock level: 1

PROBLEM

↳ ensures serializable execution.

① lost update

↳ view on conflict

② The uncommitted dependency

↳ solves: all 4 problems

③ inconsistent analysis

2) Repeatable read : lock type: 1, lock level: ϕ

④ phantom phenomenon

↳ only committed records to be read, repeated reads

of some record must return same value. However,

a transaction may not be serializable - it may

find some records inserted by a transaction but

not find others

↳ solves: problem ①, ②, ③

3) Read committed : lock type ϕ , Lock level: 1

↳ only committed records can be read, but successive reads

of record may return different (but committed) values

↳ doesn't guarantee serializability, guarantee cascadeless schedules

↳ solves: problem ②: the uncommitted dependency

4) Read uncommitted : lock type: ϕ , lock level ϕ

↳ even uncommitted records may be read, doesn't guarantee cascadeless schedule

↳ solves: none

SUBJECT: NO: DATE: / /

⑥ 3 different definitions of term transaction

1) 1960S-1970S: a transaction is a file which keeps changes to be made to the master file

2) late 70S-early 80S: a transaction is a single database operation

3) present: a transaction is a unit of program execution that accesses and possibly updates various data items

: a transaction must see a consistent database

: during transaction execution the database may be temporarily inconsistent

: When the transaction completes successfully (is committed), the database must be consistent

: In short, a transaction is a logical unit of work in which integrity constraints are allowed to be violated